

Báo cáo Kỹ thuật

Hệ thống Cảnh báo Va chạm bằng Cảm biến Siêu âm JSN-SR04T

kết nối nền tảng IoT CoreIoT

ACLAB

Ngày 4 tháng 8 năm 2026

Tóm tắt nội dung

Báo cáo này mô tả trạng thái hiện tại của hệ thống nhúng cảnh báo va chạm sử dụng cảm biến siêu âm chống nước JSN-SR04T, vi điều khiển ESP32-S3, nền tảng IoT CoreIoT (ThingsBoard) và màn hình cảm ứng LVGL. Nội dung tập trung vào phần cứng đang sử dụng, công cụ/framework và lý do lựa chọn, các API chính được gọi trong firmware, cũng như các chức năng đã hoàn thiện và hướng phát triển tiếp theo của hệ thống ở trạng thái hiện hành.

Mã nguồn: <https://github.com/YdtTran/supersonic-warning-system>

Mục lục

1	Giới thiệu và phạm vi hệ thống hiện tại	2
2	Kiến trúc hệ thống hiện hành	2
3	Phần cứng đang sử dụng	3
4	Công cụ và framework	4
5	Firmware Sensor Node	4
5.1	Kiến trúc tác vụ FreeRTOS	4
5.2	Bộ lọc khoảng cách (phân cụm + trung bình trượt mũ)	5
5.3	Kết nối CoreIoT	5
5.4	Cảnh báo còi cục bộ	5
6	Firmware Waveshare Screen	6
6.1	Kết nối mạng (ESP-IDF thuần)	6
6.2	Mô hình dữ liệu cảm biến	6
6.3	Giao diện LVGL 9.1	6
6.4	Lớp hỗ trợ phần cứng màn hình (BSP)	6
7	Cloud CoreIoT (Rule-Chain)	7

8	Chức năng đã hoàn thiện	7
9	Hướng phát triển tiếp theo	8

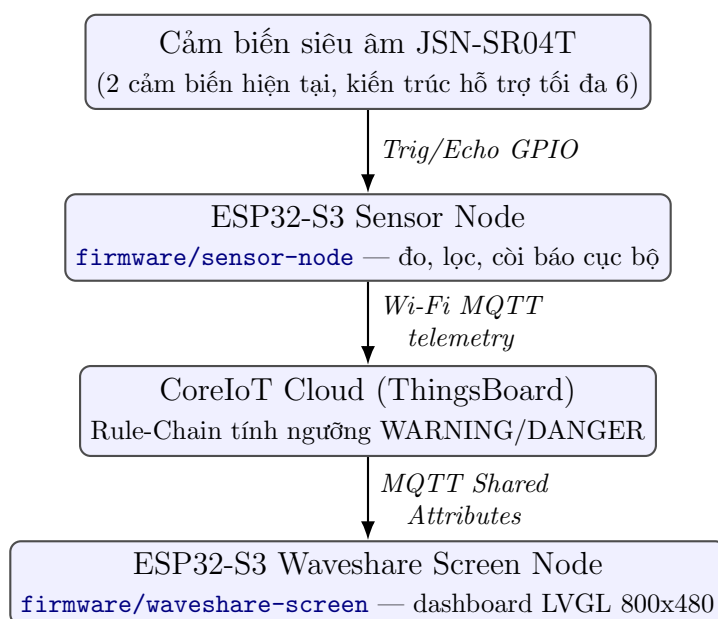
1. Giới thiệu và phạm vi hệ thống hiện tại

Hệ thống phát hiện vật cản/xe cộ ở khoảng cách gần bằng cảm biến siêu âm, gửi dữ liệu lên nền tảng đám mây CoreIoT để xử lý ngưỡng cảnh báo, sau đó định tuyến kết quả xuống màn hình hiển thị và còi cảnh báo cục bộ.

Ở trạng thái hiện tại, hệ thống đã lắp đặt và vận hành ổn định trên phần cứng thật với **2 cảm biến siêu âm** (vị trí trước-trái và trước-phải của xe). Kiến trúc phần cứng và phần mềm được thiết kế hỗ trợ tối đa **6 cảm biến** bao quanh xe; việc mở rộng lên đầy đủ 6 cảm biến hiện đang chờ bổ sung phần cứng và nằm trong hướng phát triển tiếp theo (Mục 9), không phải là giới hạn của kiến trúc.

2. Kiến trúc hệ thống hiện hành

Hình 1 mô tả luồng dữ liệu hiện hành của hệ thống: cảm biến siêu âm truyền tín hiệu Trig/Echo tới node cảm biến, node cảm biến lọc nhiễu và gửi khoảng cách qua MQTT lên CoreIoT, Rule-Chain trên CoreIoT tính toán trạng thái cảnh báo và định tuyến xuống node màn hình để hiển thị.

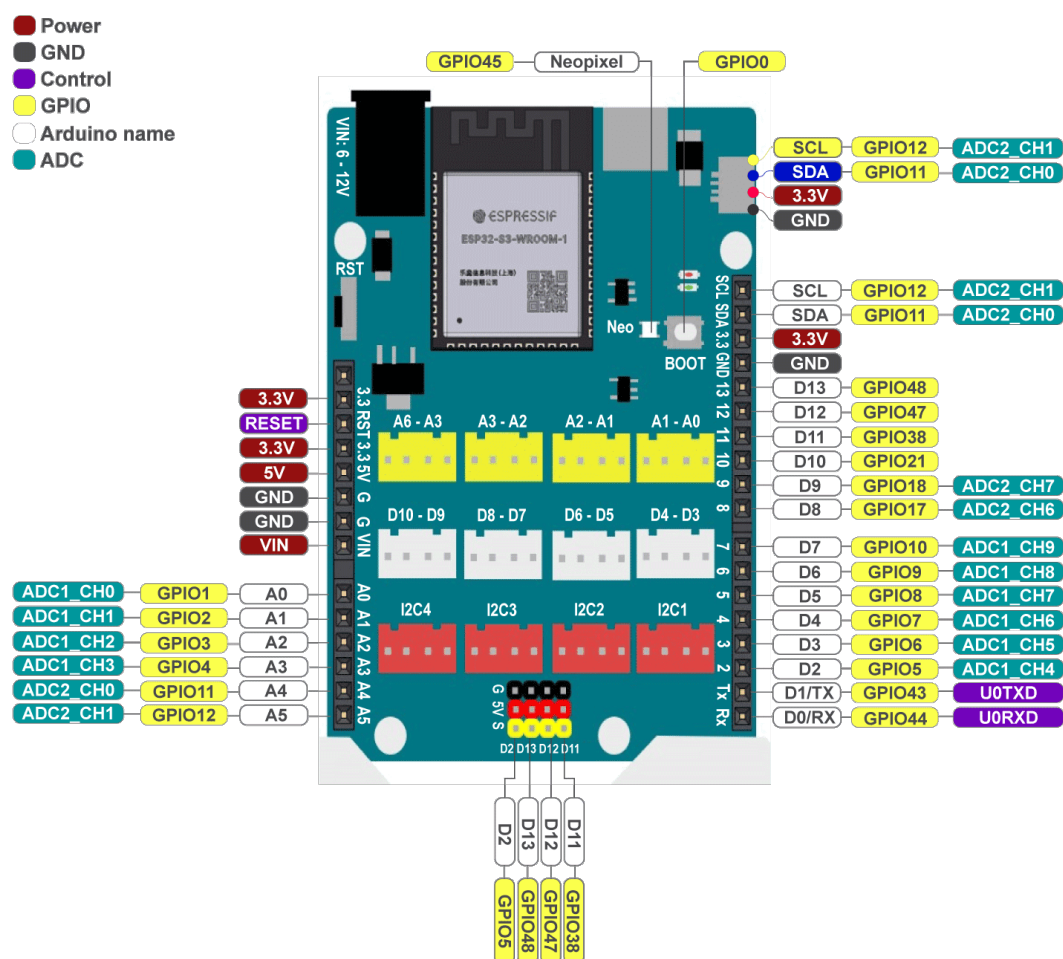


Hình 1: Luồng dữ liệu hiện hành của hệ thống.

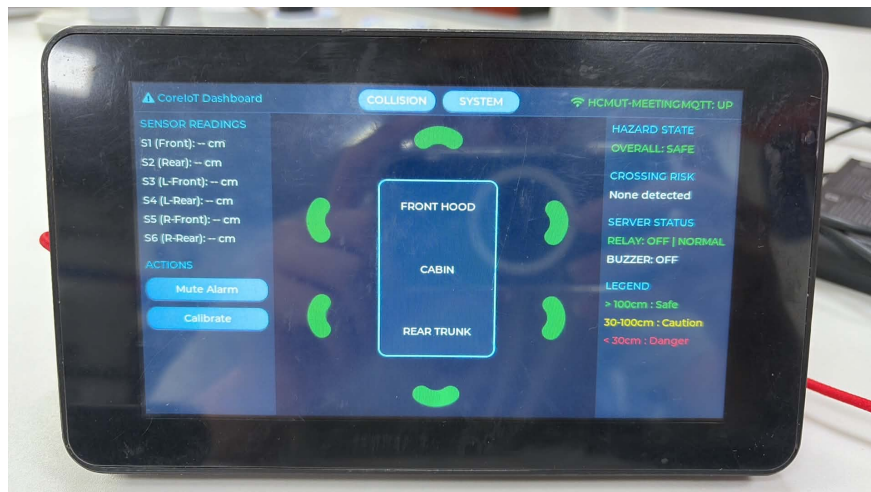
3. Phần cứng đang sử dụng

Thành phần	Mô tả
Vi điều khiển	Board Yolo:Uno , dựa trên ESP32-S3-WROOM-1 (Dual-Core 240MHz, Wi-Fi/BLE, PSRAM Octal).
Cảm biến khoảng cách	JSN-SR04T — cảm biến siêu âm chống nước, đầu dò tách rời board mạch.
Màn hình hiển thị	Waveshare ESP32-S3 Touch LCD 7" — panel RGB 800 × 480, cảm ứng dung kháng GT911 (I2C 400kHz), IO-expander CH422G (I2C, địa chỉ 0x24/0x38) điều khiển backlight, reset cảm ứng, CS thẻ SD và MUX CAN.
Cảnh báo cục bộ	Còi buzzer trên chân GPIO48 của Sensor Node, phản hồi tức thời không qua vòng lặp cloud.

Bảng 1: Danh mục phần cứng chính đang sử dụng.



Hình 2: Sơ đồ pinout board Yolo:Uno (ESP32-S3).



Hình 3: Dashboard cảnh báo va chạm chạy trên màn hình Waveshare LCD 7" (phần cứng thật).

4. Công cụ và framework

Hạng mục	Lựa chọn	Lý do
Build system	PlatformIO (<code>pio run -e yolo_uno</code>)	Layout thống nhất cho cả hai firmware, một toolchain quản lý hai dự án khác framework.
sensor-node	<code>framework = arduino</code>	Tái sử dụng hệ sinh thái thư viện Arduino-ESP32 (WiFi.h, PubSubClient) — đủ cho tác vụ đo/lọc/publish MQTT.
waveshare-screen	<code>framework = espidf</code> thuần	Driver màn hình/cảm ứng hiện đại (<code>esp_lvgl_adapter</code> , <code>esp_lcd_touch_gt911</code> , LVGL 9.1, <code>esp_lcd_panel_rgb</code>) đòi hỏi ESP-IDF ≥ 5.5 .
Đồ hoạ UI	LVGL 9.1 + <code>esp_lvgl_adapter</code>	Thư viện GUI nhúng mã nguồn mở phổ biến cho ESP32, adapter chính thức của Espressif quản lý vòng lặp render và khoá luồng an toàn.
Nền tảng IoT	CoreIoT (ThingsBoard)	MQTT-native, có sẵn Rule-Chain kéo-thả để xử lý ngưỡng cảnh báo mà không cần backend riêng.

Bảng 2: Công cụ, framework sử dụng và lý do lựa chọn.

5. Firmware Sensor Node

5.1. Kiến trúc tác vụ FreeRTOS

Bảng 3 liệt kê các tác vụ FreeRTOS đang chạy trên `sensor-node`.

Task	Stack	Priority	Core	Vai trò
SensorTask	4096	2	1	Trigger + đọc tuần tự từng cảm biến, đưa qua bộ lọc, ghi kết quả vào SharedState .
AppTask	2048	1	1	Tiêu thụ dữ liệu đã lọc song song, đọc lập chu kỳ đo.
NetworkTask	4096	1	0	Kết nối Wi-Fi/MQTT, publish khoảng cách lên CoreIoT, tách khỏi core 1 để không ảnh hưởng timing đo.

Bảng 3: Tác vụ FreeRTOS trên **sensor-node**.

5.2. Bộ lọc khoảng cách (phân cụm + trung bình trượt mũ)

Bộ lọc kết hợp phân cụm mẫu đo (loại outlier) và làm mượt bằng EMA (*Exponential Moving Average*) để giảm nhiễu vốn có của cảm biến siêu âm giá rẻ mà vẫn phản ứng nhanh với thay đổi khoảng cách thật. Bảng 4 tóm tắt các tham số chính.

Tham số	Ý nghĩa
Kích thước lịch sử	9 mẫu RAW gần nhất được lưu để phân cụm.
Cụm hợp lệ tối thiểu	5/9 mẫu phải thuộc cùng một cụm mới được chấp nhận.
Dung sai cụm	Cơ bản 8 cm, tăng theo tỉ lệ 8% so với khoảng cách đo (nhiều tỉ lệ thuận khoảng cách).
Hệ số làm mượt EMA	0.30.
Xác nhận bước nhảy	Một thay đổi lớn (>30 cm hoặc >25%) phải lặp lại 3 lần liên tiếp mới được chấp nhận, tránh nhiễu tức thời.
Reset bộ lọc	Sau 15 lần đọc lỗi liên tiếp.

Bảng 4: Tham số chính của bộ lọc khoảng cách.

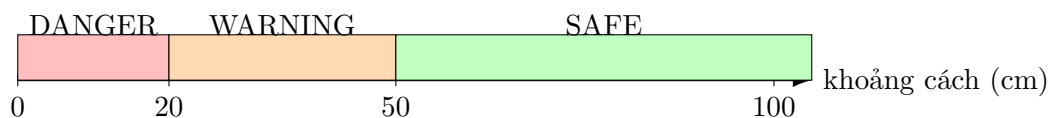
5.3. Kết nối CoreIoT

Kết nối MQTT dùng thư viện **PubSubClient** trên nền **WiFi.h** (chế độ STA), xác thực bằng access-token làm username (chuẩn ThingsBoard, không cần mật khẩu). Dữ liệu khoảng cách hai cảm biến được publish dạng JSON lên topic telemetry với tần suất 2 Hz (mã nguồn: [CoreiotClient.cpp](#)).

5.4. Cảnh báo còi cục bộ

Còi buzzer được điều khiển trực tiếp trên **sensor-node** theo hai mức ngưỡng, đồng bộ công thức với Rule-Chain phía máy chủ:

- **WARNING** (20 – 50 cm): kêu 1 lần mỗi 3 giây.
- **DANGER** (<20 cm): kêu 1 lần mỗi 1 giây.



Hình 4: Thang ngưỡng cảnh báo SAFE / WARNING / DANGER dùng chung giữa buzzer cục bộ và Rule-Chain CoreIoT.

6. Firmware Waveshare Screen

6.1. Kết nối mạng (ESP-IDF thuần)

Vì không sử dụng Arduino core, kết nối mạng dùng trực tiếp API ESP-IDF: `esp_wifi.h`, `esp_event.h`, `esp_netif.h`, component **esp-mqtt** (`mqtt_client.h`) và `nvs_flash.h`. Sau khi kết nối MQTT thành công, thiết bị subscribe topic telemetry và các topic `v1/devices/me/attributes` để nhận cập nhật trạng thái từ Rule-Chain. Dữ liệu JSON nhận được được phân tích bằng thư viện **cJSON**.

6.2. Mô hình dữ liệu cảm biến

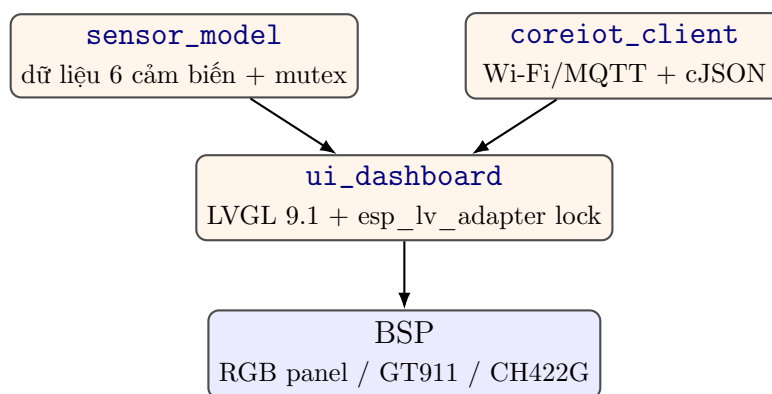
Component `sensor_model` lưu trạng thái 6 cảm biến (khoảng cách, góc lắp, cờ dữ liệu cũ), được bảo vệ bằng một mutex FreeRTOS dùng chung. Mỗi khoảng cách được phân loại thành ba vùng: **SAFE** (>100 cm), **CAUTION** (30 – 100 cm), **DANGER** (<30 cm).

6.3. Giao diện LVGL 9.1

Giao diện sử dụng các widget LVGL: `lv_arc` (biểu diễn vùng cảm biến quanh xe), `lv_label`, `lv_anim` (hiệu ứng cảnh báo), `lv_timer` (làm mới thông tin hệ thống định kỳ) và bố cục flex. Vì các tác vụ mạng chạy trên luồng (task) khác với luồng render LVGL, mọi thao tác cập nhật giao diện từ tác vụ mạng đều được bảo vệ bằng cơ chế khoá `esp_lv_adapter_lock()` với thời gian chờ giới hạn 100 ms, đảm bảo một luồng bị treo không làm treo toàn hệ thống.

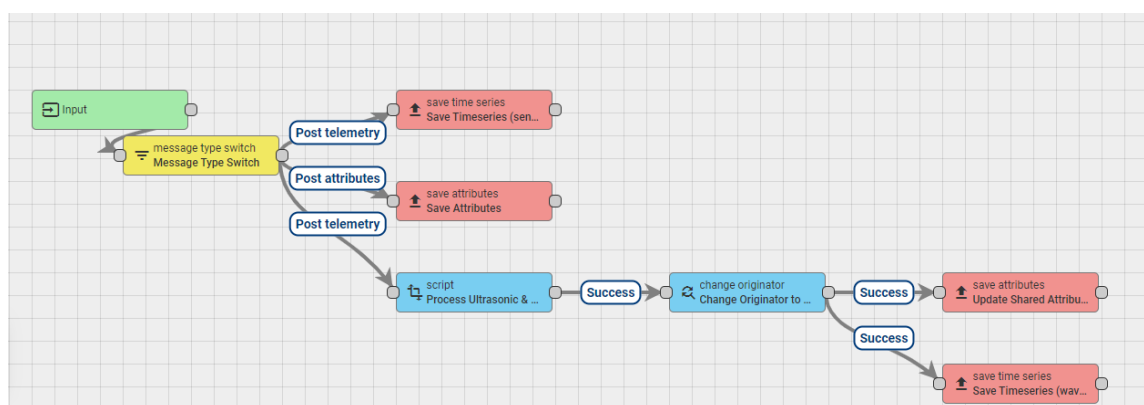
6.4. Lớp hỗ trợ phần cứng màn hình (BSP)

Panel RGB 800×480 (RGB565) được khởi tạo qua `esp_lcd_new_rgb_panel()`, framebuffer đặt trong PSRAM. Bus I2C dùng API `driver/i2c_master.h`, dùng chung cho IO-expander CH422G (ghi thanh ghi trực tiếp, không có driver riêng) và cảm ứng GT911 (driver chính thức `esp_lcd_touch_gt911`) — mã nguồn: [waveshare_rgb_lcd_port.c](#).



Hình 5: Sơ đồ khối phần mềm của waveshare-screen.

7. Cloud CoreIoT (Rule-Chain)



Hình 6: Rule-Chain CoreIoT xử lý dữ liệu telemetry và định tuyến sang node màn hình.

Rule-Chain ([supersonic_rule_chain.json](#)) nhận bản tin telemetry từ **sensor-node**, script biến đổi (JavaScript) tính khoảng cách gần nhất trong các cảm biến, suy ra trạng thái **vehicle_detected**, **warning_status** (NORMAL/WARNING/DANGER) và **relay**; sau đó đổi chủ thể bản tin sang thiết bị **waveshare-screen** và đẩy dữ liệu xuống bằng cơ chế MQTT Shared Attributes (`notifyDevice=true`) để màn hình nhận cập nhật theo thời gian thực.

8. Chức năng đã hoàn thiện

- Đo và lọc khoảng cách ổn định từ 2 cảm biến siêu âm, publish MQTT lên CoreIoT.
- Rule-Chain CoreIoT xử lý ngưỡng cảnh báo và định tuyến dữ liệu sang node màn hình.
- Dashboard LVGL hiển thị trạng thái va chạm theo thời gian thực, kèm thông tin hệ thống (Wi-Fi, trạng thái kết nối MQTT, phiên bản firmware).
- Cảnh báo còi cục bộ trên **sensor-node**, phản hồi tức thời không phụ thuộc độ trễ mạng.
- Cơ chế khoá luồng an toàn giữa tác vụ mạng và tác vụ giao diện trên node màn hình.

9. Hướng phát triển tiếp theo

- Mở rộng đủ 6 cảm biến siêu âm bao quanh xe — kiến trúc phần mềm (`sensor_model`, mảng cấu hình chân cảm biến) đã hỗ trợ sẵn, chỉ cần bổ sung phần cứng.
- Bổ sung cảnh báo cho người đi đường phía ngoài xe (còi/đèn ngoài xe).